

Employee Management and Payroll Application

Java AWT + JDBC + MySQL — Requirement Analysis, Design, Implementation Plan, Testing and Demonstration Guide

Purpose: This document converts the given lab question into a practical project plan and a report-ready answer. It explains what to develop, how the database and GUI should be designed, the main Java/JDBC logic, validation, payroll calculation, testing, and demonstration.

1. Aim

To develop a desktop-based Employee Management and Payroll Application using Java AWT for the graphical user interface, JDBC for database connectivity, and MySQL for persistent storage. The system should allow authorized HR staff to manage employee records, perform searches and CRUD operations, validate data, and calculate payroll.

2. What You Need to Develop

- Employee Registration
- Employee Search
- Employee Update
- Employee Delete
- Employee List/View
- Payroll / Salary Calculation
- Input Validation and Exception Handling
- MySQL database accessed through JDBC

3. Requirement Analysis

3.1 Functional Requirements

- Add a new employee with Employee ID, name, gender, date of birth, contact number, email, address, department, designation, date of joining, salary details and employment status.
- View and retrieve employee records.
- Search by Employee ID, department, designation or employment status.
- Update existing employee information.
- Delete existing employee records.
- Calculate gross salary and net salary.
- Prevent duplicate Employee IDs and invalid operations.
- Validate mandatory fields and numeric salary values.
- Display success and error feedback after database operations.
- Handle database and application-level exceptions.

3.2 Non-Functional Requirements

- User-friendly
- Reliable

- Accurate
- Maintainable and modular
- Consistent database operations
- Expandable for attendance, leave, performance and appraisal modules

4. Software and Hardware Requirements

- JDK (Java Development Kit)
- MySQL Server and MySQL Workbench
- MySQL Connector/J JDBC driver
- Java IDE such as Eclipse, IntelliJ IDEA or NetBeans
- A computer capable of running Java and MySQL

5. Database Design

5.1 Create Database

```
CREATE DATABASE employee_payroll;
USE employee_payroll;
```

5.2 Department Table

The department table avoids repeated department names and provides a proper relationship between departments and employees.

```
CREATE TABLE department ( department_id INT AUTO_INCREMENT PRIMARY KEY, department_name VARCHAR(100) NOT NULL
UNIQUE );
```

5.3 Employee Table

```
CREATE TABLE employee ( employee_id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, gender VARCHAR(10) NOT NULL,
date_of_birth DATE NOT NULL, contact_number VARCHAR(15) NOT NULL, email VARCHAR(100) UNIQUE, address
VARCHAR(255), department_id INT NOT NULL, designation VARCHAR(100) NOT NULL, date_of_joining DATE NOT NULL,
basic_salary DECIMAL(10,2) NOT NULL, allowances DECIMAL(10,2) DEFAULT 0, deductions DECIMAL(10,2) DEFAULT 0,
employment_status VARCHAR(20) NOT NULL, CONSTRAINT fk_employee_department FOREIGN KEY (department_id)
REFERENCES department(department_id), CONSTRAINT chk_basic_salary CHECK (basic_salary >= 0), CONSTRAINT
chk_allowances CHECK (allowances >= 0), CONSTRAINT chk_deductions CHECK (deductions >= 0) );
```

5.4 Sample Departments

```
INSERT INTO department (department_name) VALUES ('Human Resources'), ('Information Technology'), ('Finance'),
('Marketing'), ('Sales');
```

5.5 Relationship

Department (1) → Employee (Many)

One department can contain many employees. Each employee belongs to one department.

department.department_id is the primary key and employee.department_id is the foreign key.

6. Suggested Java Project Structure

```
EmployeePayrollSystem/ ■■■ DBConnection.java ■■■ Employee.java ■■■ EmployeeDAO.java ■■■ Payroll.java ■■■
EmployeeManagement.java ■■■ Main.java
```

7. JDBC Connection

```
import java.sql.Connection; import java.sql.DriverManager; import java.sql.SQLException; public class
DBCConnection { private static final String URL = "jdbc:mysql://localhost:3306/employee_payroll"; private static
final String USER = "root"; private static final String PASSWORD = "your_password"; public static Connection
getConnection() throws SQLException { return DriverManager.getConnection(URL, USER, PASSWORD); } }
```

Replace **your_password** with the password used by your MySQL installation and add MySQL Connector/J to the Java project's classpath.

8. Employee Class and OOP

```
public class Employee { private int employeeId; private String name; private String gender; private String
dateOfBirth; private String contactNumber; private String email; private String address; private int
departmentId; private String designation; private String dateOfJoining; private double basicSalary; private
double allowances; private double deductions; private String employmentStatus; public Employee() {} // Add
constructors, getters and setters for all fields. }
```

This demonstrates classes, objects and encapsulation. Private fields with public getters/setters protect the employee data. AWT event handling can also use interfaces such as ActionListener.

9. AWT GUI Design

9.1 Main Menu

```
+-----+ | EMPLOYEE MANAGEMENT & PAYROLL SYSTEM |
+-----+ | [ Register Employee ] [ Search Employee ] | | [
Update Employee ] [ Delete Employee ] | | [ View Employees ] [ Payroll ] | | [ Exit ] |
+-----+
```

9.2 Employee Registration Screen

Use Frame/Panel, Label, TextField, Choice, Button, TextArea and Dialog. Fields should include Employee ID, Name, Gender, Date of Birth, Contact Number, Email, Address, Department, Designation, Date of Joining, Basic Salary, Allowances, Deductions and Employment Status.

9.3 Search Screen

Provide a Choice or buttons for search type: Employee ID, Department, Designation or Employment Status. Display matching records in a TextArea.

9.4 Update and Delete

For update, obtain the Employee ID, retrieve the existing record, allow editing and save changes. For delete, confirm the Employee ID exists, ask for confirmation, then delete the record.

9.5 Employee List/View

Retrieve employees with a JOIN to the department table and display important fields such as ID, name, department, designation, salary and employment status.

10. CRUD SQL

10.1 Insert

```
INSERT INTO employee (employee_id, name, gender, date_of_birth, contact_number, email, address, department_id,
designation, date_of_joining, basic_salary, allowances, deductions, employment_status) VALUES (?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?);
```

10.2 Retrieve / Search

```
SELECT e.*, d.department_name FROM employee e JOIN department d ON e.department_id = d.department_id WHERE
e.employee_id = ?;
```

Similar queries can search by department, designation or employment_status using parameterized PreparedStatements.

10.3 Update

```
UPDATE employee SET name=?, contact_number=?, email=?, address=?, designation=?, basic_salary=?, allowances=?,
deductions=?, employment_status=? WHERE employee_id=?;
```

10.4 Delete

```
DELETE FROM employee WHERE employee_id = ?;
```

11. Payroll Module

Gross Salary = Basic Salary + Allowances

Net Salary = Gross Salary – Deductions

```
public double calculateGrossSalary(double basic, double allowances) { return basic + allowances; } public
double calculateNetSalary(double gross, double deductions) { return gross - deductions; }
```

Example: Basic Salary = ■30,000; Allowances = ■5,000; Deductions = ■2,000. Gross Salary = ■35,000 and Net Salary = ■33,000.

12. Validation

- Mandatory fields must not be empty.

- Employee ID must be numeric and positive.
- Employee ID must be unique.
- Salary, allowances and deductions must be numeric and non-negative.
- Update and delete must reject non-existing Employee IDs.
- Email/contact fields can be checked for appropriate formats.

```
try { double salary = Double.parseDouble(salaryField.getText().trim()); if (salary < 0) { showMessage("Salary cannot be negative"); return; } } catch (NumberFormatException ex) { showMessage("Invalid salary value"); }
```

13. Exception Handling

```
try { Connection con = DBConnection.getConnection(); // database operation } catch (SQLException ex) { // database error } catch (NumberFormatException ex) { // invalid numeric input } catch (Exception ex) { // general application error }
```

14. Testing

Test Case	Input	Expected Result
New employee	Valid details	Employee inserted
Duplicate employee	Existing Employee ID	Error displayed
Search	Existing ID	Employee details displayed
Search	Non-existing ID	Employee not found
Update	Existing ID + valid data	Employee updated
Update	Non-existing ID	Employee not found
Delete	Existing ID	Employee deleted
Delete	Non-existing ID	Employee not found
Empty field	Name empty	Validation error
Invalid salary	abc	Invalid salary message
Negative salary	-5000	Salary rejected
Payroll	30000, 5000, 2000	Gross 35000; Net 33000
DB failure	MySQL unavailable	Connection error handled

15. OOP Concepts for Viva

Encapsulation: Employee attributes are private and accessed using getters/setters.

Class and Object: Employee is a class; an Employee object stores one employee's information.

Inheritance: A GUI class can extend AWT Frame.

Polymorphism: Event handling can implement interfaces such as ActionListener and provide its actionPerformed implementation.

Abstraction: DAO methods can hide database implementation details from the GUI layer.

16. Recommended Development Order

1. Install JDK, MySQL, MySQL Workbench and MySQL Connector/J.
2. Create the employee_payroll database and tables.
3. Create the Java project and packages/classes.
4. Test JDBC connection.
5. Implement Employee model.
6. Implement INSERT, SELECT, UPDATE and DELETE using PreparedStatement.
7. Create the AWT main menu and individual screens.
8. Connect GUI buttons to DAO methods.
9. Add validation and exception handling.
10. Implement payroll calculation.
11. Run all test cases and capture screenshots.
12. Prepare the final demonstration and report.

17. Final Demonstration Checklist

- Show MySQL database and tables.
- Show the running Java AWT application.

- Register a new employee.
- Search the employee.
- Update employee information.
- View the employee list.
- Calculate payroll.
- Try duplicate Employee ID and show validation.
- Try empty mandatory field and show validation.
- Try invalid salary and show validation.
- Try searching/deleting a non-existing employee.
- Demonstrate database connection failure handling if required.

18. Future Enhancements

- Attendance management
- Leave management
- Performance tracking
- Employee appraisal
- Login/authentication and role-based access
- Payslip generation
- Advanced reporting

19. Conclusion

The proposed Employee Management and Payroll Application provides a structured solution for maintaining employee information and payroll-related data. Java AWT provides the desktop user interface, JDBC connects the application to MySQL, SQL manages persistent records, OOP provides modularity, and validation and exception handling improve reliability. The modular design also makes it possible to add attendance, leave, performance and appraisal features in the future without redesigning the entire application.